

Data Generation Addendum 1

Addendum to: `Data_Generation_Methodology.pdf` (original design lock) **Trigger:** Findings from `02_profiling.sql`, Section 3 (referential integrity) **Status:** Applied. Data regenerated. Verified against original design targets.

Why this exists as an addendum, not an edit to the original

The original methodology document is a pre-generation design lock: every parameter was specified before any data existed, precisely so the design couldn't be reverse-justified after the fact. Editing that document after generation would erase that property. Instead, issues found during profiling are logged here, with what was found, why it happened, what changed, and proof the fix worked, without altering the historical record.

Issue 1: Claims generated outside the member's enrollment window

Found in: Profiling Section 3.3 (referential integrity between claims and membership)

Symptom: 324,140 claim rows (31.95% of all claims) belonged to a member with no membership record for that specific service month. A member who joined the plan in month 30 could still be assigned a claim in month 5, before they existed in the plan.

Root cause: the generator selected which member received a claim, and separately and independently selected which month the claim occurred in, using the same year-over-year trend curve for every member regardless of when that member was actually enrolled. The two draws were never checked against each other.

Why this was a real bug, not intentional dirtiness: every other data quality issue in Section 5 of the original methodology was injected at a small, stated, realistic rate (1 to 4%). A claim physically cannot occur for coverage that does not yet exist. A 32% rate of this is not realistic messiness, it is a modeling error in the generator.

Fix: claim service month is now drawn only from the assigned member's own `[enroll_start, enroll_end]` range (the same range used to build `membership_raw.csv`), still weighted by the original year-over-year trend curve, but restricted to months that member was actually enrolled in.

Verification after the fix, full 1,014,550-row generation:

Metric	Before fix	After fix
Claims outside the member's enrollment window	31.95%	3.48%

The residual 3.48% is not a new problem. It is the expected consequence of the already-documented, intentional ~3.5% coverage-gap injection in `membership_raw.csv` (Section 5 of the original methodology, "missing enrollment month"). A member whose membership row was deliberately dropped for a gap month will still show as "outside enrollment" for claims in that month, which is correct behavior, not a defect.

Issue 2: Service dates never fell on the 29th, 30th, or 31st

Found in: Profiling Section 1.1 (`last_service_date` = 2024-12-28, never later)

Root cause: the day-of-month offset was drawn from a fixed range of 0 to 27 regardless of the actual length of that month, so days 29 to 31 could never be generated.

Fix: the day-of-month offset is now drawn using each month's actual number of days (28, 29, 30, or 31, correctly varying by month), so every real calendar day is reachable.

Verification after the fix: day-of-month range is now 1 to 31.

Confirmation that the core design was not affected

Both fixes changed only *which day and month* a claim's `service_date` falls on. Neither touched cost generation, so the design in Sections 4 and 5 of the original methodology was re-checked after regeneration to confirm it still holds:

Design target (original methodology)	Re-verified after fix
Inpatient ~8-10% of volume	10%
Inpatient drives ~65% of dollars	68%
Top 1% of claims ≈ 20-25% of dollars	Consistent with original profiling

Row counts (unchanged)

Regeneration used the same target sizes as the original design. Row counts after the fix are effectively identical to the original run (both draw from the same generation targets; minor differences are normal across any two random generations):

File	Rows
<code>claims_raw.csv</code>	1,014,550
<code>membership_raw.csv</code>	1,214,303

What this means for anything already built on the first data version

Nothing was built downstream of the first generation before this fix was found (staging was reloaded fresh with the corrected data). No rework was required beyond re-running `01_staging.sql` against the new CSVs.